

Equipo A (Kappa) — Primera Entrega Técnica

Contenido

- 1. Introducción..... 2
- 2. Arquitectura Kappa 2
- 3. Diagrama de arquitectura 3
- 4. Diagrama de flujo 3
- 5. Tabla de responsabilidades 3

1. Introducción

El objetivo de esta entrega es presentar:

1. El diagrama de arquitectura de la solución basada en **Arquitectura Kappa**.
2. El **primer avance técnico**, incluyendo:
 - Ingesta inicial en NiFi
 - Topics base en Kafka
 - Estructura inicial para Flink y PySpark
 - Configuración inicial del entorno y APIs

La arquitectura propuesta unifica datos históricos y datos en tiempo real en un **único flujo**, cumpliendo la definición de Kappa: un único pipeline de eventos desde Kafka que Flink procesa de forma continua.

2. Arquitectura Kappa

La arquitectura diseñada se basa en un log único de eventos, donde tanto los datos históricos (batch) como los datos de streaming se convierten en eventos homogéneos que se publican en Kafka.

Componentes principales

1. **Fuentes de datos:**
 - CSV históricos:
 - o delays_history_agg.csv
 - o delays_history_sample.csv
 - o weather_history.csv
 - APIs:
 - o OpenSky (vuelos en tiempo real)
 - o OpenWeather (meteorología)
2. **NiFi (Ingesta unificada):**
 - Lectura de CSV históricos
 - Llamadas periódicas a OpenSky y OpenWeather
 - Conversión a JSON estandarizado
 - Publicación en Kafka (topic único: events_all)
 - Rutas de error (/data/errors/...)
3. **Kafka (Log de eventos único):**

Topic principal:

 - events_all → contiene:
 - o eventos históricos
 - o eventos de streaming

4. Flink (Stream Processing):

Pipeline único:

- Ventanas temporales
- Join vuelo-meteorología
- Cálculo de KPIs
- Escritura de:
 - KPIs tiempo real → MongoDB/Cassandra
 - Resultados históricos → MySQL/HDFS

5. Power BI:

- Vista histórica (MySQL)
- Vista real-time (MongoDB/Cassandra)

3. Diagrama de arquitectura

El diagrama de arquitectura básicamente muestra cómo funciona todo nuestro sistema de datos siguiendo el modelo Kappa.

Diagrama de arquitectura

4. Diagrama de flujo

El diagrama de flujo sirve para explicar paso a paso cómo se mueve la información por dentro del sistema, desde que entra hasta que se procesa y se guarda. Básicamente es la “ruta” que siguen los datos.

Diagrama de flujo

5. Tabla de responsabilidades

Miembro	Tareas realizadas en esta entrega
Hugo	Configuración APIs, .env.example, estructura inicial del repo, documentación, diagrama de arquitectura
Alex	Configuración Docker, Kafka topics, entorno base
Nacho	Primer flujo NiFi, ingestión CSV, rutas de error